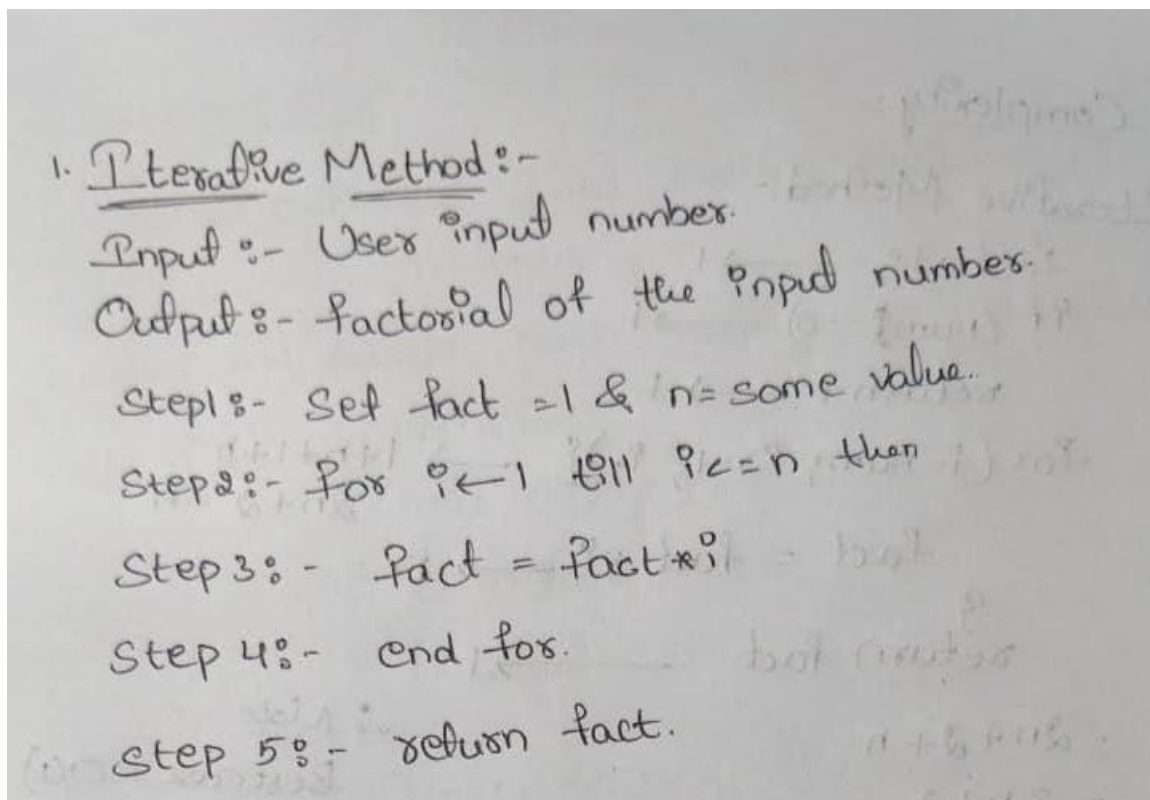


Practical:4

Aim:

Implementation and Time analysis of factorial program using Iterative and Recursive method.

1.Iterative Method Alogrithm:



Program:

```
#include<iostream>
using namespace std;
float fact(float n){
    int fact = 1;
    if (n==1){
```

```
    return 1;
}
else{
    for(int i=1;i<=n;i++){
        fact*=i;
    }
    return fact;
}

}

int main(){
    int n;
    cout<<"enter the value of n:";
    cin>>n;
    int a = fact(n);
    cout<<"The factorial of"<<n<<"is:"<<a;
    return 0;
}
```

Output:

```
Enter the value of n: 7
The factorial of 7 is: 5040
```

Time Complexity:

Time Complexity:-

1. Iterative Method:-

fact = 1, $\rightarrow 1$

if (num == 0) $\rightarrow 1$

return 1 $\rightarrow 1$

for (i = num; i >= 1; i--) { $\rightarrow 1 + n + 1 + n$
 $2n + 2 = n$

fact = fact * i $\rightarrow n$

}
return fact $\rightarrow 1$

$$= 2n + 2 + n$$

$$= 3n + 2$$

$$= n$$

The Time complexity is $O(n)$.

Note

Best case. = $O(n)$

Avg case. = $O(n)$

Worst case = $O(n)$

2. Recursive Method Algorithm:

2. Recursive method:-

Input :- User input number.

Output :- Factorial number of input number.

Step 1:- Start.

Step 2:- Read value of n .

Step 3:- Factorial(n)

Step 4:- Check if $n \leftarrow 0$ or $n \leftarrow 1$

Return 1

Step 5:- else.

$n * \text{factorial}(n-1)$

Step 6:- Print factorial

Step 7:- Stop.

Program:

```
#include<iostream>

using namespace std;

int fact_(int n){
    int fact =1;
    if (n==0 || n==1){
        return 1;
    }
    else{
        fact = n*fact_(n-1);
    }
}

int main(){
    int n;
    cout<<"Enter the value of n:";
    cin>>n;
    int a = fact_(n);
    cout<<"Enter the factorial of "<<n<<" is:"<<a;
    return 0;
}
```

Output:

```
Enter the value of n: 7
The factorial of 7 is: 5040
```

Time Complexity:

2. Recursive Method:-

```

int fact (int num) {
    if (num == 0)
        return 1;
    while (num >= 1) {
        fact = fact * num;
        num--;
    }
    return fact;
}

```

Time complexity is $O(n)$

Calculation:
 $= 3n + 4$
 $= n$

Note:-
 Best case:- $O(n)$
 Avg case:- $O(n)$
 Worst case:- $O(n)$

Conclusion:

The factorial program was successfully implemented using both iterative and recursive methods. Both methods produce the same result, but the recursive method uses function calls and stack space. The time complexity of both methods is $O(n)$, making them efficient for calculating factorials.